LAPORAN PRAKTIKUM MIKROPROSESOR DAN INTERFACE

KONVERSI BILANGAN DESIMAL KE HEKSADESIMAL, HEKSADESIMAL KE OKTAL DAN KONVERSI BINER TO GRAY – GRAY TO BINER

Disusun untuk memenuhi tugas praktikum Mikroprosesor Dan Interface

Oleh

Gaitsha Azzahra Devitrya

231344012

Tanggal Percobaan: 27 Mei & 1 Juni 2025

Tanggal Penyerahan: 5 Juni 2025

Instruktur: 1. Ferry Satria, BSEE., M.T.

2. Rahmawati Hasanah, S.ST., M.T.

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2025

DAFTAR ISI

DAFTAR ISI.....ii
A. TUJUAN.....2
B. LANDASAN TEORI.....2
C. ALAT DAN BAHAN.....3
D. LANGKAH KERJA.....3
E. HASIL PERCOBAAN DAN ANALISIS DATA.....4
G. KESIMPULAN.....13
DAFTAR PUSTAKA.....14

A. TUJUAN

1. Mahasiswa dapat memahami konsep dan mampu membuat program assembly untuk konversi bilangan desimal ke heksadesimal.
2. Mahasiswa dapat memahami konsep dan mampu membuat program assembly untuk konversi bilangan heksadesimal ke oktal.
3. Mahasiswa dapat memahami konsep dan mampu membuat program assembly untuk konversi bilangan gray ke biner ataupun konversi bilangan biner ke gray.

B. LANDASAN TEORI

a. Konversi Bilangan Desimal ke Heksadesimal

Konversi bilangan desimal ke heksadesimal merupakan salah satu teknik penting dalam pemrograman komputer dan sistem digital, karena sering digunakan untuk merepresentasikan alamat memori, warna, serta instruksi mesin. Bilangan desimal berbasis 10 menggunakan digit 0 hingga 9, sedangkan bilangan heksadesimal menggunakan basis 16, yang melibatkan digit 0 hingga 9 dan huruf A hingga F, di mana A merepresentasikan 10 dalam desimal, B untuk 11, hingga F yang mewakili 15.

Langkah-langkah yang dilakukan adalah dengan membagi bilangan desimal dengan 16 secara berulang hingga hasil bagi menjadi nol, sambil mencatat sisa setiap pembagian. Sisa pembagian pertama menjadi digit paling kanan (least significant digit), sedangkan sisa pembagian terakhir menjadi digit paling kiri (most significant digit

b. Konversi Bilangan Heksadesimal ke Oktal

Konversi bilangan heksadesimal ke oktal adalah proses mengubah bilangan dengan basis 16 (heksadesimal) ke bilangan dengan basis 8 (oktal). Cara yang umum digunakan adalah dengan terlebih dahulu mengonversi setiap digit heksadesimal ke bentuk biner 4-bit, kemudian mengelompokkan digit biner tersebut menjadi kelompok 3-bit dari kanan ke kiri, dan akhirnya mengubah setiap kelompok 3-bit tersebut ke bentuk oktal. Dengan cara ini, konversi dari heksadesimal ke oktal dapat dilakukan secara sistematis dan akurat.

c. Konversi bilangan Biner ke Gray dan Gray ke Biner

Gray code adalah sistem penomoran biner dimana dua nilai yang bersebelahan hanya memiliki tepat satu digit beda. Gary code digunakan untuk memfasilitasi koreksi error pada komunikasi digital. Konversi bilangan biner ke bilangan Gray adalah proses mengubah bilangan biner menjadi representasi Gray agar transisi antara angka-angka digital menjadi lebih stabil, karena hanya satu bit yang berubah pada setiap kenaikan nilai. Konversi ini dilakukan dengan cara mempertahankan bit pertama sama seperti bit pertama biner, sedangkan setiap bit berikutnya diperoleh dengan melakukan operasi XOR antara bit sebelumnya pada biner dan bit saat ini pada biner.

Sebaliknya, konversi dari Gray ke biner dilakukan dengan cara menyalin bit pertama sama seperti bit pertama Gray, lalu untuk setiap bit berikutnya, dilakukan operasi XOR antara bit sebelumnya pada hasil biner dan bit sekarang pada Gray.

d. Metode yang digunakan dalam program konversi bilangan

1. Metode Look Up Table

Metode ini menggunakan tabel statis (array) yang berisi hasil konversi untuk setiap digit atau nilai tertentu. Saat program ingin melakukan konversi, ia langsung mengambil nilai hasil konversi dari tabel ini berdasarkan indeks digit yang dicari.

2. Metode Tabel puluhan

Metode ini membuat tabel yang berisi nilai digit pangkat atau posisi dalam sistem bilangan tertentu (misalnya 16^0, 16^1, 16^2 untuk Hex atau 8^0, 8^1, 8^2 untuk Octal). Proses konversinya menggunakan pembagian dengan nilai pangkat yang sesuai.

3. Metode counter

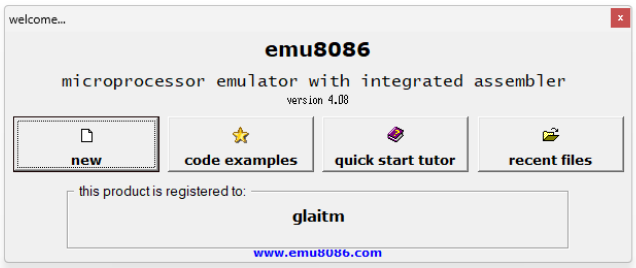
Metode ini menggunakan perhitungan secara iteratif (looping). Untuk desimal ke Hex misalnya, program melakukan pembagian secara berulang (divide and modulo) untuk memperoleh digit paling rendah, lalu menyimpan digitnya dan melanjutkan dengan pembagian berikutnya.

C. ALAT DAN BAHAN

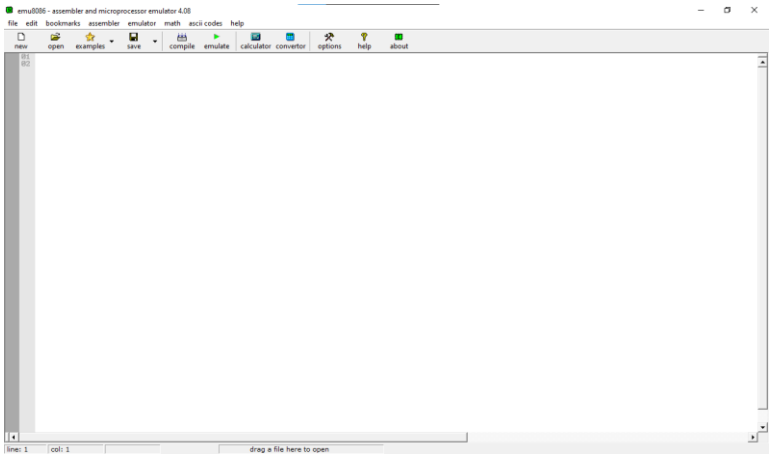
1. Laptop/PC
2. Software EMU 8086

D. LANGKAH KERJA

1. Instal Software emu8086 pada laptop dan buka aplikasinya
2. Setelah membuka aplikasi maka akan muncul tampilan *new* pada monitor

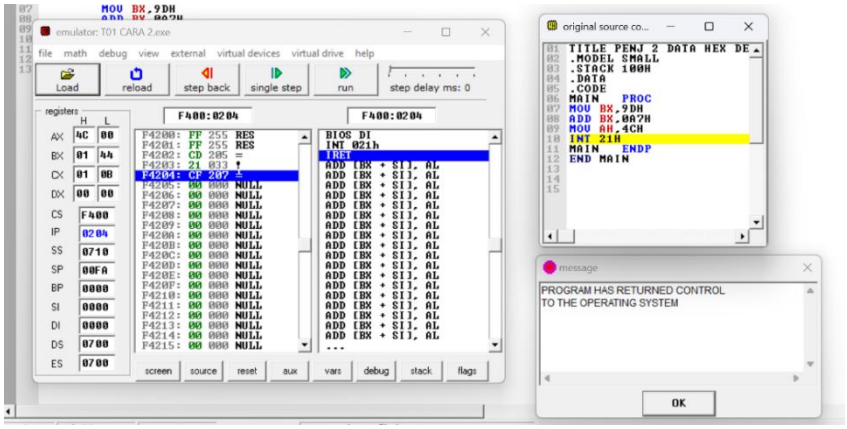


3. Maka setelah itu akan tampil worksheet seperti gambar dibawah ini :



Worksheet ini digunakan untuk menuliskan kode.

4. Untuk memeriksa kode yang berada di worksheet apakah program sudah benar atau belum, klik “compile”
5. Jika sudah disave maka jika program sudah benar akan ada tulisan “run”.
6. Jika sudah di ”run” maka akan muncul hasilnya pada monitor



E. HASIL PERCOBAAN DAN ANALISIS DATA

1. Percobaan Praktikum 1 (Konversi bilangan Desimal ke Heksa dan Heksa ke Oktal)

Pada praktikum ini terdapat 6 soal yang diberikan dan dapat dianalisis sebagai berikut :

1) Buat program untuk mengubah data desimal 2 digit menjadi heksadesimal. Gunakan metode tabel data *(look up table)*. Gunakan register CH sebagai input dan register DH sebagai output

Program :

```
TITLE KONVERSI ;2 DIGIT DES -> HEKSA
.MODEL  SMALL
.STACK  100H
.DATA
    TBL1 DB 0H, 1H, 2H, 3H, 4H, 5H, 6H, 7H, 8H, 9H
    TBL2 DB 0H, 0AH, 14H, 1EH, 28H, 32H, 3CH, 46H, 50H, 5AH
.CODE

 MAIN PROC
      MOV AX, @DATA
      MOV DS, AX
 ; 4BIT BAWAH /LOW NIBEL
      MOV BL,CH
      AND BL,0FH

      MOV BH,0
      MOV SI,BX
      MOV BL,[TBL1+SI]

 ;HIGH NIBEL, 4BIT ATAS
      MOV AL,CH
      AND AL,0F0H
      SHR AL,4
      MOV AH,0
      MOV SI,AX
      MOV DH,[TBL2+SI]

      ADD DH,BL


      MOV AH,4CH
      INT 21H
MAIN ENDP
     END  MAIN
```

Hasil Program :



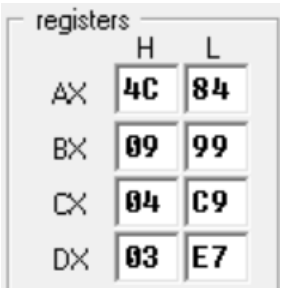
2) Buat program untuk mengubah data desimal 3 digit menjadi heksadesimal menggunakan metode tabel data *(look up table)*. Gunakan register BX sebagai input dan register DX sebagai output.

Program :

```
TITLE DESIMAL=> HEKSA 3 DIGIT LOOK UP TABLE
.MODEL SMALL
.STACK 100
.DATA
 TABEL1 DB 0H,1H,2H,3H,4H,5H,6H,7H,8H,9H                    ;SATUAN
 TABEL2 DB 0H,0AH,14H,1EH,28H,32H,3CH,46H,50H,5AH           ;PULUHAN
 TABEL3 DW 0H,64H,0C8H,12CH,190H,1F4H,258H,2BCH,320H,384H   ;RATUSAN
.CODE
 MAIN PROC
      MOV AX,@DATA
      MOV DS,AX
      ;UNTUK DIGIT SATUAN
      MOV BX,999H
      MOV AL,BL
      AND AL,0FH
      MOV AH,0
      MOV SI,AX
      MOV AL,[TABEL1+SI]
      MOV AH,0
      MOV DX,AX
      ;UNTUK DIGIT PULUHAN
      MOV AL,BL
      AND AL,0F0H
      SHR AL,4
      MOV AH,0
      MOV SI,AX
      MOV AL,[TABEL2+SI]
      MOV AH,0
      ADD DX,AX
      ;UNTUK DIGIT RATUSAN
      MOV AL,BH
      AND AL,0FH
      MOV AH,0
      MOV SI,AX
      ADD SI,SI
      MOV AX,[TABEL3+SI]
      ;MENGGABUNGKAN SATUAN DAN PULUHAN
      ADD DX,AX
      ADC CX,DX

      MOV AH,4CH
      INT 21H
MAIN  ENDP
      END MAIN
```

Hasil Program :



3) Buat program untuk mengubah data desimal 4 digit menjadi heksadesimal menggunakan metode tabel data *(look up table)*. Gunakan register BX sebagai input dan register CX sebagai output.

Program :

```
TITLE KONVERSI ;4DIGIT DESIMAL -> HEKSA
.MODEL SMALL
.STACK 100H
.DATA
TBL1 DB 0,1H,2H,3H,4H,5H,6H,7H,8H,9H,10H,11H,12H,13H,14H,15H
TBL2 DB 0,0AH,14H,1EH,28H,32H,3CH,46H,50H,5AH
TBL3 DW 0,64H,0C8H,12CH,190H,1F4H,258H,2BCH,320H,384H
TBL4 DW 0,3E8H,7D0H,0BB8H,0FA0H,1388H,1770H,1B58H,1F40H,2328H
.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    MOV CX,0
    MOV AL,BL
    AND AL,0FH
    MOV AH,0
    MOV SI,AX
    MOV CL,[TBL1+SI]

    MOV AL,BL
    AND AL,0F0H
    SHR AL,4
    MOV AH,0
    MOV SI,AX
    ADD CL,[TBL2+SI]
    ADC CH,0

    MOV AL,BH
    AND AL,0FH
    MOV AH,0
    ADD AX,AX
    MOV SI,AX
    ADD CX,[TBL3+SI]

    MOV AL,BH
    AND AL,0F0H
    SHR AL,4
    MOV AH,0
    ADD AX,AX
    MOV SI,AX
    ADD CX,[TBL4+SI]

    MOV AH,4CH
    INT 21H
MAIN ENDP
```

Hasil Program :



4) Buat program untuk mengubah data heksadesimal 2 digit yang ada pada register CH menjadi oktal. Simpan hasil konversi pada register DL.

Program :

```
TITLE KONVERSI HEKSA 2 DIGIT (CH) -> OKTAL (DX)
.MODEL SMALL
.STACK 100H
.DATA
    TBL1 DB 0H,1H,2H,3H,4H,5H,6H,7H,10H,11H,12H,13H,14H,15H,16H,17H
    TBL2 DW 0H,20H,40H,60H,100H,120H,140H,160H,200H,220H,240H,260H,300H,320H,34
.CODE
MAIN    PROC
        MOV AX, @DATA
        MOV DS, AX

        MOV CH, 0FFH
        MOV BL, CH
        AND BL, 0FH
        ;MOV BH, 0

        MOV AL, BL
        MOV AH, 0
        MOV SI, AX
        MOV BL, [TBL1 + SI]

        MOV BH, CH
        AND BH, 0F0H
        SHR BH, 4
        MOV DL, BH
        MOV DH, 0

        ADD DX, DX
        MOV SI, DX
        MOV DX, [TBL2 + SI]

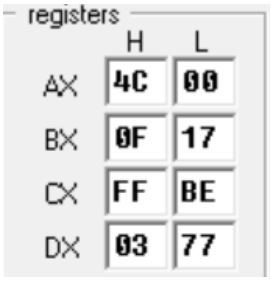
        MOV AL, BL
        ADD DL, AL
        CALL SDAA

        MOV AL, 0
        ADC DH, AL
        CALL SDAA

        MOV AH,4CH
        INT 21H
MAIN    ENDP
```

```
SDAA PROC   ; SUBRUTIN SDAA
    PUSH BX
    PUSH CX
    MOV BH,AH
    LAHF
    MOV BL,AH
    MOV CH,AL
    MOV CL,AL
    AND CL, 0FH; CL=SATUAN
    ;==============
    TEST AH,10H
    JZ L22
    ADD AL,8
    JNC L22
    ADD AL,80H
    OR BL,1
L22: TEST AH,1
    JZ L33
    ADD AL,80H
    OR BL,1
L33: MOV CH,AL
    AND CH,0FH
    CMP CH,7
    JBE L44
    ADD AL,8
    JNC L44
    ADD AL,80H
    OR BL,1
L44: MOV CH,AL
    AND CH,0F0H
    CMP CH,70H
    JBE L55
    ADD AL,80H
    OR BL,1
L55: MOV AH,BL
    SAHF
    POP CX
    POP BX
    MOV AH,BH
    RET
SDAA ENDP
        END MAIN
```

Hasil Program :



5) Buat program untuk mengubah data heksadesimal 4 digit yang ada pada register CX menjadi oktal. Simpan hasil konversi pada register CL:DX.

Program :

```
TITLE KONVERSI ;4 DIGIT HEXA KE OKTAL CX->CL:DX
.MODEL SMALL
.STACK 100H
.DATA
DAT   DB 0, 1, 2, 3, 4, 5, 6, 7, 10H, 11H, 12H, 13H, 14H, 15H, 16H, 17H
DAT2 DW 0, 20H, 40H, 60H, 100H, 120H, 140H, 160H, 200H, 220H, 240H,
      DW 260H, 300H, 320H, 340H, 360H
DAT3 DW 0, 400H, 1000H, 1400H, 2000H, 2400H, 3000H, 3400H, 4000H
      DW 4400H, 5000H, 5400H, 6000H, 6400H, 7000H, 7400H
DAT4 DW 0, 0, 0, 1H, 0, 2H, 0, 3H, 0, 4H, 0, 5H, 0, 6H
      DW 0H, 7H, 0, 10H, 0, 11H, 0, 12H, 0, 13H, 0, 14H
      DW 0, 15H, 0, 16H, 0, 17H
.CODE
MAIN PROC
      MOV AX, @DATA
      MOV DS, AX
      MOV BL, CL
      AND BL, 0FH
      MOV BH, 0
      MOV SI, BX
      MOV BL, [DAT+SI]

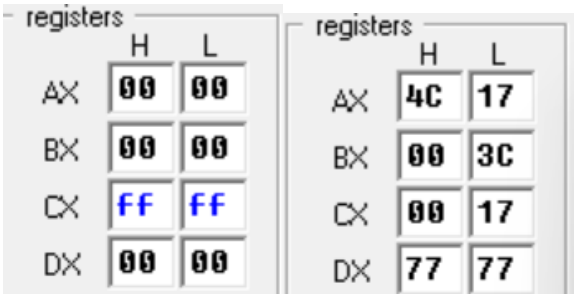
      MOV AL, CL
      AND AL, 0F0H
      SHR AL, 4
      MOV AH, 0
      ADD AX, AX
      MOV SI, AX
      MOV DX, [DAT2+SI]

      ADD DX, BX
      CALL SDAA

      MOV AL, CH
      AND AL, 0FH
      MOV AH, 0
      ADD AX, AX
      MOV SI, AX
      ADD DX, [DAT3+SI]
      CALL SDAA
      ADC CL, 0
      MOV AL, CH
      AND AL, 0F0H
      SHR AL, 4
      MOV AH, 0
      ADD AX, AX
      MOV SI, AX
      ADD BX, AX
      ADD BX, AX
      ADD BX, AX
      MOV SI, BX
      MOV CX, 0
      ADD DX, [DAT4+SI]
      CALL SDAA
      ADC CX, 0
      ADD CX, [DAT4+SI+2]
      CALL SDAA


      MOV AH, 4CH
      INT 21H
MAIN ENDP
SDAA PROC   ; SUBRUTIN SDAA
      PUSH BX
      PUSH CX
      MOV BH,AH
      LAHF
      MOV BL,AH
      MOV CH,AL
      MOV CL,AL
      AND CL, 0FH; CL=SATUAN
      ;==============
      TEST AH, 10H
      JZ L22
      ADD AL, 8
      JNC L22
      ADD AL, 80H
      OR BL,1
L22: TEST AH,1
      JZ L33
      ADD AL,80H
      OR BL,1
L33: MOV CH,AL
      AND CH,0FH
      CMP CH,7
      JBE L44
      ADD AL,8
      JNC L44
      ADD AL,80H
      OR BL,1
L44: MOV CH,AL
      AND CH,0F0H
      CMP CH,70H
      JBE L55
      ADD AL,80H
      OR BL,1
L55: MOV AH,BL
      SAHF
      POP CX
      POP BX
      MOV AH,BH
      RET
SDAA ENDP
END MAIN
```

Hasil program :



6) Buat program untuk mengubah data heksadesimal 6 digit yang ada pada BM1menjadi oktal. Simpan hasil konversi pada BM2.

Program :

```
TITLE KONVERSI ; HEKSA -> OKTAL 6 DIGIT(BM1->BM2)
.MODEL SMALL
.STACK 100H
.DATA
BM1 DW 0AFAFH,23H,0FF00H,11H,0AAAAH,0C1H,0BBBBH,17H,0DDDDH,0A1H,0EEAAH,0F2H
BM2 DW 30 DUP(0)
.CODE
MAIN PROC
      MOV AX, @DATA
      MOV DS, AX

      MOV SI, 0
      MOV DI, 0

      MOV AX, 0
      MOV BX, 0
      MOV CX, 0
      MOV DX, 0
 LUP:MOV CX, [BM1+SI]
      MOV BX, [BM1+SI+2]

      MOV DX, CX
      MOV AX, BX
      AND DL, 0F8H
      AND DH, 0FFH
      AND AL, 0FFH
      AND AH, 0FFH
      ADD DL, CL
      ADC DH, CH
      ADC AL, BL
      ADC AH, BH

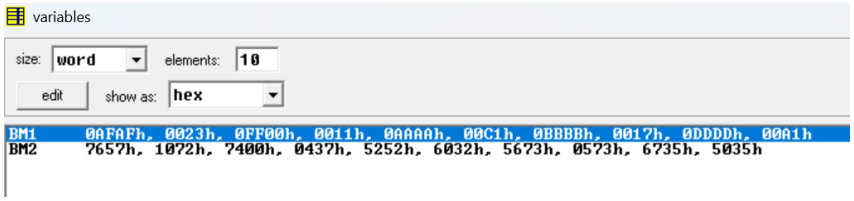

      MOV CX, 0
      MOV BX, DX
      MOV CX, AX
      AND BL, 80H
      AND BH, 0FFH
      AND CL, 0FFH
      AND CH, 0FFH
      ADD BL, DL
      ADC BH, DH
      ADC CL, AL
      ADC CH, AH
      MOV AX, 0
      MOV DX, 0
      MOV DX, BX
      MOV AX, CX
      AND DL, 00H
      AND DH, 00H
      AND AL, 0F8H
      AND AH, 0FFH
      ADD DL, BL
      ADC DH, BH
      ADC AL, CL
      ADC AH, CH

      MOV BX, 0
      MOV CX, 0
      MOV BX, DX
      MOV CX, AX
      AND BL, 00H
      AND BH, 00H
      AND CL, 80H
      AND CH, 0FFH
      ADD BL, DL
      ADC BH, DH
      ADC AL, CL
      ADC AH, CH

      MOV [BM2+SI], DX
      MOV [BM2+SI+2], AX
      MOV AX, 0
      MOV AX, SI
      ADD AL, 4H
      MOV SI, AX
      MOV AX, DI
      ADD AL, 4
      MOV DI, AX
      CMP DI, 24H
      JNE LUP

      MOV AH, 4CH
      INT 21H
MAIN ENDP
END MAIN
```

Hasil Program :



Analisis Data :

Konversi bilangan desimal ke heksadesimal :

Percobaan praktikum ini yang pertama yaitu membuat program konversi bilangan desimal ke heksadesimal dengan metode look-up table (LUT) ini menggunakan teknik tabel yang berisi nilai-nilai heksadesimal dari setiap digit desimal 0 hingga 9. Dengan memanfaatkan tabel, program bisa mengambil hasil konversi secara langsung tanpa harus melakukan perhitungan yang kompleks seperti pembagian atau perkalian.

Untuk konversi dua digit desimal (00–99), program membagi angka menjadi dua bagian, yaitu digit rendah (4 bit bawah) dan digit tinggi (4 bit atas). Digit rendah diambil dengan instruksi AND untuk memisahkan 4 bit bawah, kemudian digunakan sebagai indeks untuk mengambil nilai dari tabel LUT1. Sementara digit tinggi diambil dengan melakukan operasi shift (SHR) agar posisinya menjadi 4 bit bawah, lalu diakses melalui tabel LUT2. Hasil konversi dari kedua digit tersebut kemudian dijumlahkan sehingga membentuk angka heksadesimal yang sesuai.

Kemudian pada program konversi 3 digit (000–999) atau 4 digit (0000–9999) desimal, prinsipnya sama, yaitu dengan memecah setiap digit (ribuan, ratusan, puluhan, satuan) lalu mengambil hasilnya dari tabel. Seperti, pada 3 digit desimal, digit ratusan, puluhan, dan satuan dipisahkan menggunakan instruksi AND dan shift, lalu masing-masing digit dikonversi melalui tabel look up dan disusun ulang ke posisi yang sesuai. Untuk digit ribuan pada 4 digit desimal, langkahnya serupa dan inputan dimasukan pada CX dan hasilnya dilihat pada register DX.

Konversi bilangan Heksadesimal ke Oktal:

Percobaan kedua pada praktikum ini yaitu membuat progrma asseblt untuk mengonversi bilangan heksadesimal menjadi bilangan oktal dengan menggunakann metode look-up table.

Pada program 2 digit heksadesimal (CH → DX), pertama-tama bilangan heksa yang ingin dikonversi dimasukkan ke register CH. Selanjutnya, nilai CH dipecah menjadi dua bagian, yaitu 4 bit bawah (low nibble) dan 4 bit atas (high nibble). Setiap bagian kemudian diakses melalui tabel look-up (TBL1 dan TBL2) yang sudah ada di segmen data. Nilai-nilai di tabel ini sudah berisi representasi oktal dari setiap digit heksa (0-Fh). Nilai dari tabel kemudian diambil menggunakan indeks (SI) dan ditambahkan ke DX untuk menyusun hasil konversi dalam register DX. Dengan metode ini, proses konversi tidak memerlukan perhitungan manual yang panjang, cukup memanfaatkan indexing pada tabel.

Untuk konversi bilangan heksa 4 digit (CX → CL:DX), logikanya mirip namun dilakukan dua kali lipat. Program bekerja dengan memanfaatkan teknik masking (AND) dan shifting (SHR) untuk memecah nilai heksadesimal yang ada di CX menjadi 4 nibble (masing-masing 4 bit). Setelah setiap nibble diambil, program menggunakan tabel look-up yang sudah disiapkan di bagian

data (TBL1 dan TBL2) untuk mendapatkan nilai representasi oktalnya. Setiap hasil konversi nibble ke oktal ini kemudian dikombinasikan dan disusun kembali dengan memanfaatkan register DX untuk bagian hasil bawah (empat digit oktal) dan register CL untuk bagian hasil atas (dua digit oktal), sehingga register CL:DX menyimpan hasil konversi bilangan oktal secara lengkap. Program juga memanfaatkan subrutin SDAA yang membantu menggabungkan nilai oktal yang telah dipecah per nibble ke dalam satu output yang lebih lengkap.

Dan pada soal terakhir yaitu soal 6 merupakan program assembly untuk mengonversi bilangan Heksadesimal 6 digit ke bilangan Oktal. Progam ini dibuat dengan menginputkan datanya pada segment data BM1 dan nantinya akan ditampilkan hasilnya pada vars/segment data BM2. Data heksadesimal disusun dalam pasangan byte untuk membentuk bilangan 16-bit yang akan diproses satu per satu.

Pertama , program menginisialisasi pointer SI untuk iterasi data dan register-register lainnya seperti BX, CX, dan DX yang digunakan sebagai buffer sementara. Loop utama LUP: bertugas mengambil dua byte dari array BM dan menggabungkannya menjadi satu bilangan 16-bit (dengan mengakses BM[SI] dan BM[SI+1], serta menyimpannya dalam DX).

Setelah data 16-bit berhasil diambil, dilakukan serangkaian operasi pergeseran (shift) dan masking (AND dengan 0Fh, 03h, FFh, dll.) untuk mengekstrak grup bit 3-bit yang merupakan dasar dari sistem bilangan oktal. Hal ini dilakukan dengan cara menggeser nilai DX ke kanan (secara logis) dan menambahkan hasilnya ke register AX melalui operasi ADD dan ADC (Add with Carry) untuk menjaga ketepatan hasil konversi, termasuk bit-bit carry dari operasi sebelumnya.

Bagian akhir loop mengisi array BM2 dengan hasil konversi dari bilangan heksadesimal ke bilangan oktal yang sudah dikalkulasi dan disimpan dalam AX. Penulisan ke BM2 dilakukan dalam dua bagian (menyimpan low byte dan high byte hasil). Selanjutnya, nilai indeks SI dan DI diperbarui dan dibandingkan dengan batas (dalam hal ini 24h atau 36 desimal, menunjukkan jumlah byte dalam array BM), dan program akan melanjutkan loop sampai seluruh data dikonversi.

2. Percobaan Praktikum 2 (Konversi bilangan Biner to Gray dan Gray to Biner)

Pada praktikum ini terdapat 4 soal yang diberikan dan dapat dianalisis sebagai berikut :

1) Buat program untuk mengubah data Biner 8-bit yang berada pada register CH menjadi kode Gray. Simpan hasil konversi pada register CL

Program :

```
TITLE KONVERSI  ;BINER -> GREY 8-BIT
.MODEL SMALL
.STACK 100H
.DATA

.CODE
MAIN PROC

      ;INPUT NILAI KE CH
      MOV CL,CH

      ;GESER KANAN 1X
      SHR CL,1

      ;XOR DGN NILAI ASLINYA
      XOR CL,CH ;HASIL DI DH


      MOV AH, 4CH
      INT 21H
MAIN ENDP
END MAIN
```

Hasil Program:



2) Buat program untuk mengubah data Gray 8-bit yang berada pada register CH menjadi Biner. Simpan hasil konversi pada register CL

Program :

```
TITLE KONVERSI ;GRAY -> BINER 8-BIT
.MODEL SMALL
.STACK 100H
.DATA

.CODE
MAIN PROC

      ;ISIKAN NILAI KE CH
      MOV DH,CH
      MOV DL,7 ;UNTUK PENGULANGAN/COUNTER
      RCL DH,1

      ;KRN BUTUH NILAI CF
      ;MAKA PUSH UNTUK MENGAMANKAN NILAI CF
      PUSHF      ;MENYIMPAN NILAI FLAG KE-N KEDALAM STACK

      ;MASUKAN NILAI CF
      ;BERAPUN NILAI CL DI BIT PALING KANAN
      ;AKAN MASUK NILAI CF
      RCL CL,1

      ;MENGUJI BIT KE-2 DARI KIRI SAMPAI  BIT KE-7
      L1: RCL DH,1
      JNC L3
      POPF        ;MENAGMBIL NILAI FLAG KE-N DARI STACK
      ;KOMPLEMENKAN
      CMC

      L2: PUSHF  ;MENGUPDATE NILAI FLAG (NILAI FLAG KE-(N+1)
      RCL CL,1
      DEC DL
      JNZ L1 ;COUNTER SELESAI

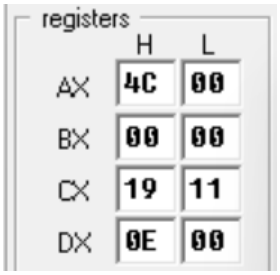

      MOV AH, 4CH
      INT 21H

       ;MENGAMBIL NILAI FLAG KE-N DARI STACK
      L3: POPF ;MANGGIL LAGI
      ;SESUAI KONSEPAN TERDETEKSI NILAI 0/MENGCOPY

      JMP L2 ;JUMP KE L2 UNTUK MASUKKIN NILANYA KE CL

MAIN ENDP
END MAIN
```

Hasil Program:



3) Buat program untuk mengubah data Biner 16-bit yang berada pada register CX menjadi Gray. Simpan hasil konversi pada register DX

Program :

```
01 TITLE KONVERSI BINER -> GREY 16-BIT (CX->DX)
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 .CODE
06 MAIN PROC
07
08     MOV AX, CX
09     MOV DX, CX
10
11     SHR AX, 1
12     XOR DX, AX
13
14     MOV AH, 4CH
15     INT 21H
16
17 MAIN ENDP
18 END MAIN
```

Hasil Program :

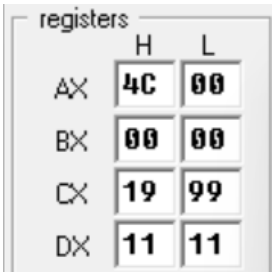


4) Buat program untuk mengubah data Gray 16-bit yang berada pada register CX menjadi Biner. Simpan hasil konversi pada register DX

Program :

```
01 TITLE KONVERSI GREY -> BINER 16-bit  (CX->DX)
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05
06 .CODE
07 MAIN PROC
08
09      MOV BX,CX
10      MOV AL,15
11
12      RCL BX,1
13      PUSHF
14      RCL DX,1
15
16 L1:  RCL BX,1
17      JNC L3
18      POPF
19      CMC
20
21 L2:  PUSHF
22      RCL DX,1
23      DEC AL
24      JNZ L1
25
26      MOV AH, 4CH
27      INT 21H
28
29 L3:  POPF
30
31      JMP L2
32
33 MAIN ENDP
34 END MAIN
```

Hasil Program :



Analisis Data :

Pada praktikum ini membuat program konversi bilangan biner ke gray dan gray ke biner baik untuk bilangan 8-bit maupun 16-bit dalam bahasa asssembly. Bilangan ini dapat dikonversi dengan memanfaatkan operasi logika yaitu EX-OR maupun penggunakan pergeseran bit.

Pada program konversi biner ke gray (untuk 8-bit dan 16-bit), prinsip yang digunakan adalah bit pertama (MSB) gray sama dengan bit pertama biner, sedangkan bit-bit berikutnya diperoleh dengan operasi XOR antara bit biner saat ini dengan bit biner sebelumnya yang telah digeser satu posisi ke kanan.

Pada program 8-bit, nilai biner diambil dari register CH kemudian dilakukan operasi SHR CL,1 (geser kanan satu bit) untuk mendapatkan bit biner sebelumnya. Setelah itu, hasilnya dioperasikan XOR dengan nilai aslinya XOR CL, CH, yang secara langsung memberikan hasil gray di register CL.

Pada program 16-bit, prinsip yang sama digunakan, tetapi dengan register yang lebih lebar (CX dan DX). Nilai awal disalin ke AX dan DX, lalu SHR AX, 1 dan diikuti XOR DX, AX untuk memperoleh hasil gray di DX. Dengan demikian, perbedaan antara versi 8-bit dan 16-bit hanya terletak pada panjang data yang digunakan (1-byte atau 2-byte), sedangkan logikanya tetap sama: geser dan XOR.

Kemudian, konversi gray ke biner (8-bit dan 16-bit) memerlukan perhitungan iteratif di mana bit pertama biner sama dengan bit pertama gray, dan bit-bit setelahnya dihitung dengan melakukan XOR antara bit gray saat ini dengan hasil biner sebelumnya. Konversi ini lebih kompleks daripada biner ke gray karena proses nya harus berantai.

Pada program 8-bit, nilai gray disimpan di register CH. Bit MSB langsung sama (karena bit pertama biner = bit pertama gray), sedangkan bit-bit setelahnya dihitung dengan melakukan operasi XOR berulang terhadap hasil biner sebelumnya. Ini diwujudkan dalam kode melalui penggunaan flag carry dan stack (PUSHF, POPF) untuk menyimpan dan memproses carry secara berulang (looping) menggunakan instruksi RCL dan CMC. Program secara iteratif memproses bit demi bit, mulai dari bit kedua hingga bit ke-7, dengan mengupdate hasil biner secara dinamis.

Pada program 16-bit, konsep yang sama diterapkan tetapi dengan register yang lebih besar (CX, BX, DX) dan loop yang lebih panjang (bit ke-

0 hingga bit ke-15). Penggunaan RCL, PUSHF, POPF, dan CMC menjadi krusial untuk memindahkan bit secara tepat dan mengelola carry flag agar nilai biner yang dihasilkan sesuai.

F. KESIMPULAN

Pada praktikum ini dapat disimpulkan bahwa praktikan dapat lebih memahami cara kerja software EMU8086 yang dapat menjalankan program yang digunakan untuk mikroprosesor. Praktikan juga lebih memahami bagaimana cara membuat program mengonversi bilangan dari desimal ke heksadesimal juga konversi bilangan heksadesimal ke oktal. Terdapat 3 metode yang dipelajari, yaitu metode perkalian berulang, look up table dan cara alternatif. Namun pada praktikum ini, praktikan hanya mengimplementasikan 2 cara yaitu look up table dan pergeseran yang dimana kedua metode tersebut berhasil diimplementasikan sehingga praktikan berhasil membuat program untuk mengonversi bilangan heksadesimal ke desimal yang akhirnya ditampilkan bisa di register maupun di vars atau array.

Pada praktikum kedua,dapat disimpulkan bahwa praktikan juga mempelajari cara membuat program assembly untuk mengonversi bilangan Biner ke Gray maupun sebaliknya. Membuat program konversi dengan bilangan 8-bit dan 16-bit memberikan pemahaman yang baik pada praktikan untuk membuat program dan juga mengenal instruksi" baru. Secara keseluruhan, kedua praktikum ini membuat praktikan memiliki pemahaman baru dan kemampuan dalam membuat program assemblu yang dijalankan pada *software* EMU 8086.

DAFTAR PUSTAKA

Arief Wisaksono, I. M. (2019). *Buku Ajar Mikroprosesor* . Retrieved from eprint.umsida.ac.id:http://eprints.umsida.ac.id/7199/1/31.%20buku%20ajar%20mikroprosessor%20komplate%20-%20Arief%20Wisaksono.pdf